

Lesson 02 Demo 01

Implementing an AutoGen Agent Chain for Automated Code Testing and Debugging

Objective: To build an AutoGen and Azure OpenAI-based agent chain where a CodeGen agent writes code, a tester agent generates tests, and a debugger agent fixes failing code until tests pass or the loop exits, automating code quality assurance (QA) and reducing manual debugging effort

Tools required: Visual Studio Code, AutoGen (AG2) with Azure OpenAI API

Prerequisites: Basic familiarity with Python functions and the idea of agent roles

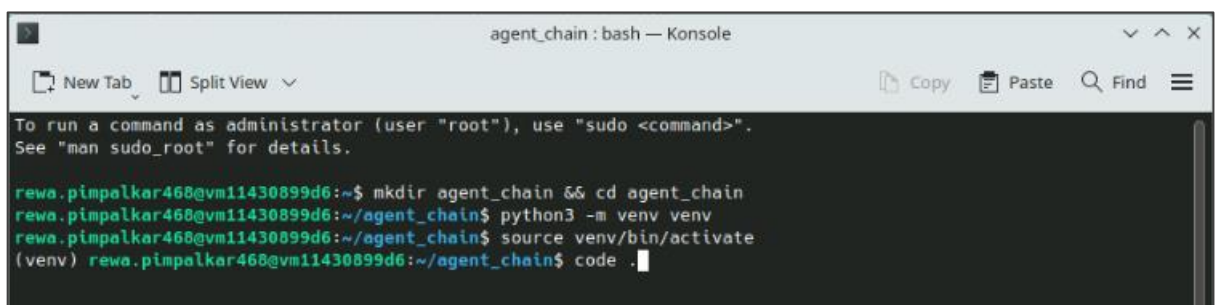
Steps to be followed:

1. Set up the local environment in Visual Studio Code
2. Configure the Azure OpenAI model for AutoGen
3. Define agent roles and helper functions
4. Implement and run the orchestrator logic

Step 1: Set up the local environment in Visual Studio Code

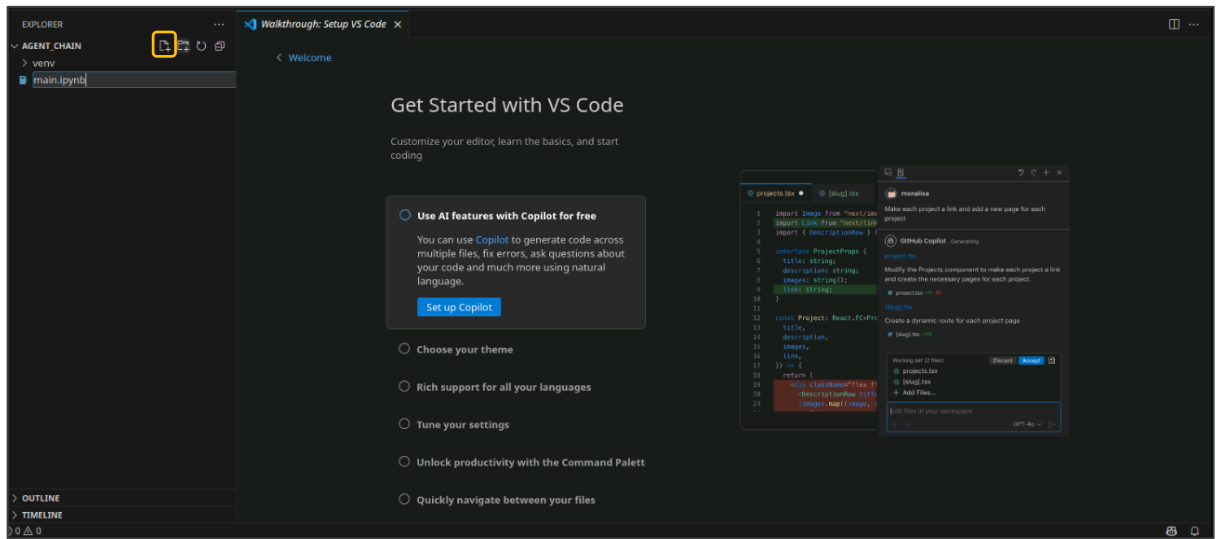
- 1.1 Open the terminal and run the following commands to create and open your project workspace:

```
mkdir agent_chain && cd agent_chain
python3 -m venv venv
source venv/bin/activate
code .
```



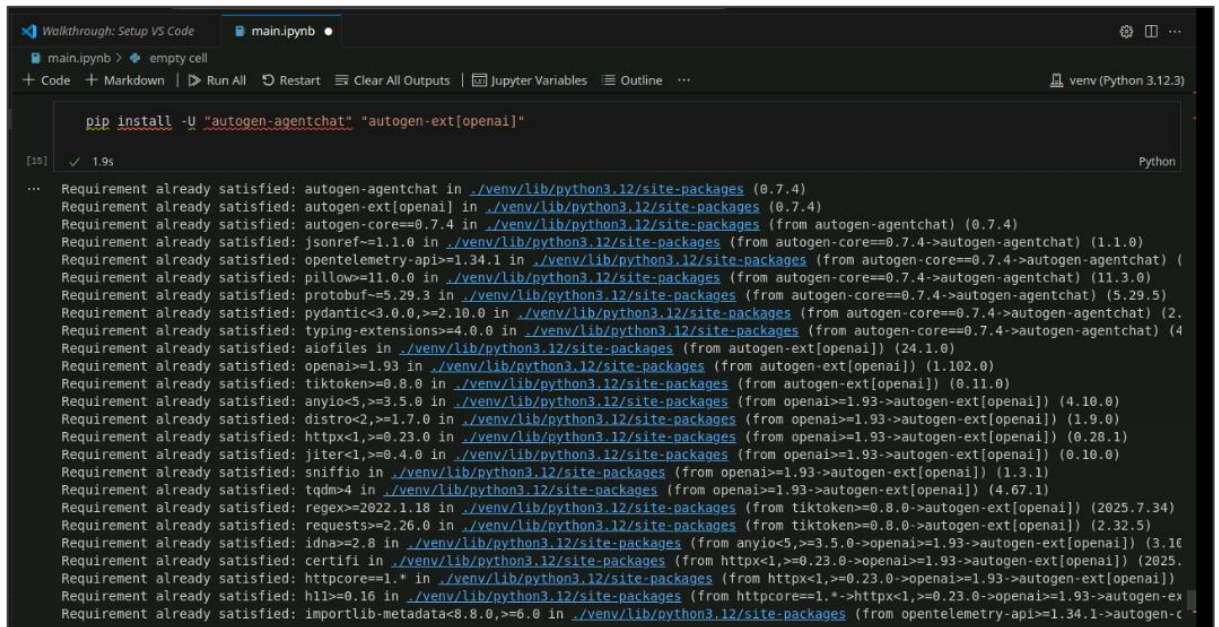
```
agent_chain : bash — Konsole
New Tab Split View
Copy Paste Find
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
rewa.pimpalkar468@vm11430899d6:~$ mkdir agent_chain && cd agent_chain
rewa.pimpalkar468@vm11430899d6:~/agent_chain$ python3 -m venv venv
rewa.pimpalkar468@vm11430899d6:~/agent_chain$ source venv/bin/activate
(venv) rewa.pimpalkar468@vm11430899d6:~/agent_chain$ code .
```

1.2 Click the **New File...** icon and create a new file named **main.ipynb**



1.3 Execute the following command to install the required libraries that enable the CodeGen, tester, and debugger agents to work with Azure OpenAI:

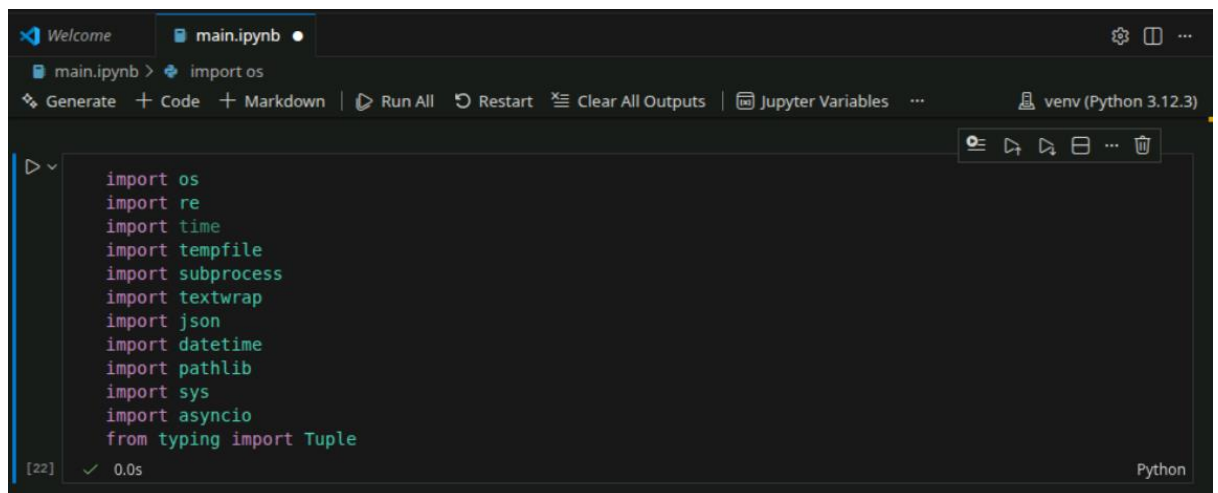
pip install -U "autogen-agentchat" "autogen-ext[openai]"



Step 2: Configure the Azure OpenAI model for AutoGen

2.1 Import the standard Python libraries needed for logging, file handling, test execution, and code formatting using the following code:

```
import os
import re
import time
import tempfile
import subprocess
import textwrap
import json
import datetime
import pathlib
import sys
import asyncio
from typing import Tuple
```

A screenshot of a Jupyter Notebook interface. The top bar shows 'Welcome' and 'main.ipynb'. Below the top bar, there's a toolbar with icons for 'Generate', '+ Code', '+ Markdown', 'Run All', 'Restart', 'Clear All Outputs', 'Jupyter Variables', and a dropdown for 'venv (Python 3.12.3)'. The main area is a code cell with the following Python code:

```
import os
import re
import time
import tempfile
import subprocess
import textwrap
import json
import datetime
import pathlib
import sys
import asyncio
from typing import Tuple
```

 The cell is labeled '[22]' and shows a green checkmark and '0.0s' indicating successful execution. The bottom right corner says 'Python'.

2.2 Import the ConversableAgent class and configure Azure OpenAI (API key, endpoint, model, and parameters) so agents can connect and run using the following code:

```
# AutoGen (AG2) import
from autogen_agentchat.agents import AssistantAgent
from autogen_agentchat.messages import TextMessage
from autogen_ext.models.openai import AzureOpenAIChatCompletionClient

# Azure OpenAI configuration
```

```
API_KEY =
"2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bz
fXJ3w3AAAABACOGi3sC"
os.environ["AZURE_OPENAI_API_KEY"] = API_KEY

MODEL = "gpt-5-mini" # deployment name
llm_cfg = {
    "config_list": [
        {
            "api_type": "azure",
            "model": MODEL,
            "api_key": API_KEY,
            "base_url": "https://openai-api-management-gw.azure-api.net",
            "api_version": "2025-01-01-preview",
        }
    ]
}

azure_cfg = llm_cfg["config_list"][0]

model_client = AzureOpenAIChatCompletionClient(
    azure_endpoint=azure_cfg["base_url"],
    api_key=azure_cfg["api_key"],
    api_version=azure_cfg["api_version"],
    model=azure_cfg["model"],
    azure_deployment=azure_cfg["model"],
)
```

```

# AutoGen (AG2) import
from autogen_agentchat.agents import AssistantAgent
from autogen_agentchat.messages import TextMessage
from autogen_ext.models.openai import AzureOpenAIChatCompletionClient

# Azure OpenAI configuration
API_KEY = "2ABecnfzxhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC"
os.environ["AZURE_OPENAI_API_KEY"] = API_KEY

MODEL = "gpt-5-mini" # deployment name
llm_cfg = {
    "config_list": [
        {
            "api_type": "azure",
            "model": MODEL,
            "api_key": API_KEY,
            "base_url": "https://openai-api-management-gw.azure-api.net",
            "api_version": "2025-01-01-preview",
        }
    ]
}

```

```

        "api_key": API_KEY,
        "base_url": "https://openai-api-management-gw.azure-api.net",
        "api_version": "2025-01-01-preview",
    ]
}

azure_cfg = llm_cfg["config_list"][0]

model_client = AzureOpenAIChatCompletionClient(
    azure_endpoint=azure_cfg["base_url"],
    api_key=azure_cfg["api_key"],
    api_version=azure_cfg["api_version"],
    model=azure_cfg["model"],
    azure_deployment=azure_cfg["model"],
)

```

[23] ✓ 0.0s

Step 3: Define agent roles and helper functions

3.1 Execute the following code to set up rate limits, logging, and helper functions for extracting code, running tests, and handling agent replies:

Light rate limit and retry caps

SLEEP_BETWEEN_CALLS_SEC = float(os.environ.get("RATE_SLEEP_SEC", "1.0"))

MAX_DEBUG_ROUNDS = int(os.environ.get("MAX_DEBUG_ROUNDS", "2"))

Logging and run artifacts

```

RUN_DIR = pathlib.Path("runs") /
datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
RUN_DIR.mkdir(parents=True, exist_ok=True)
LOG_FILE = RUN_DIR / "log.jsonl"

def log_event(kind: str, payload: dict):
    rec = {"ts": datetime.datetime.now(datetime.timezone.utc).isoformat(), "kind":
kind, **payload}
    with LOG_FILE.open("a", encoding="utf-8") as f:
        f.write(json.dumps(rec, ensure_ascii=False) + "\n")

# Helpers
CODE_BLOCK_RE = re.compile(r"```(?:python)?\s*(.*?)```", re.DOTALL |
re.IGNORECASE)

def extract_code_block(text: str) -> str:
    m = CODE_BLOCK_RE.search(text or "")
    return textwrap.dedent(m.group(1).strip()) if m else textwrap.dedent(text or
 "").strip()

def run_py_tests(solution_code: str, test_code: str, timeout_sec: int = 10) ->
Tuple[bool, str]:
    with tempfile.TemporaryDirectory() as tmp:
        sol = pathlib.Path(tmp) / "solution.py"
        tst = pathlib.Path(tmp) / "test_solution.py"
        sol.write_text(solution_code, encoding="utf-8")
        tst.write_text(test_code, encoding="utf-8")

        cmd = [sys.executable, str(tst)]
        try:
            proc = subprocess.run(cmd, capture_output=True, text=True,
timeout=timeout_sec, cwd=tmp)
            out = (proc.stdout or "") + (proc.stderr or "")
            passed = proc.returncode == 0
            return passed, out.strip()
        except subprocess.TimeoutExpired as e:
            return False, f"TIMEOUT after {timeout_sec}s\n{e}"
        except Exception as e:
            return False, f"EXEC_ERROR: {e}"

async def _ask_async(agent: AssistantAgent, message: str, label: str) -> str:

```

```

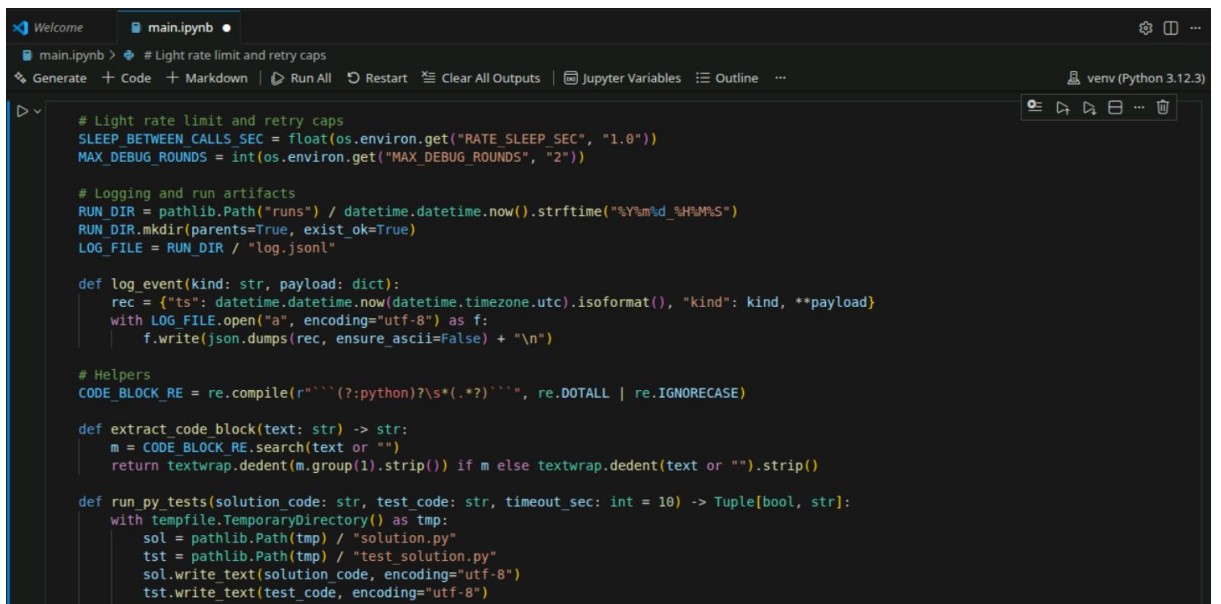
await asyncio.sleep(SLEEP_BETWEEN_CALLS_SEC)
result = await agent.on_messages(
    messages=[TextMessage(content=message, source="user")],
    cancellation_token=None,
)

content = ""
if hasattr(result, "chat_message") and getattr(result.chat_message, "content",
None) is not None:
    content = result.chat_message.content
elif hasattr(result, "messages") and result.messages:
    content = getattr(result.messages[-1], "content", "") or ""
else:
    content = str(result)

log_event("llm_reply", {"agent": getattr(agent, "name", "assistant"), "label":
label, "content": content})
return content

def ask(agent: AssistantAgent, message: str, label: str) -> str:
    return asyncio.run(_ask_async(agent, message, label))

```



```

# Light rate limit and retry caps
SLEEP_BETWEEN_CALLS_SEC = float(os.environ.get("RATE_SLEEP_SEC", "1.0"))
MAX_DEBUG_ROUNDS = int(os.environ.get("MAX_DEBUG_ROUNDS", "2"))

# Logging and run artifacts
RUN_DIR = pathlib.Path("runs") / datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
RUN_DIR.mkdir(parents=True, exist_ok=True)
LOG_FILE = RUN_DIR / "log.jsonl"

def log_event(kind: str, payload: dict):
    rec = {"ts": datetime.datetime.now(datetime.timezone.utc).isoformat(), "kind": kind, **payload}
    with LOG_FILE.open("a", encoding="utf-8") as f:
        f.write(json.dumps(rec, ensure_ascii=False) + "\n")

# Helpers
CODE_BLOCK_RE = re.compile(r'```(?:python)?\s*(.*?)```', re.DOTALL | re.IGNORECASE)

def extract_code_block(text: str) -> str:
    m = CODE_BLOCK_RE.search(text or "")
    return textwrap.dedent(m.group(1).strip()) if m else textwrap.dedent(text or "").strip()

def run_py_tests(solution_code: str, test_code: str, timeout_sec: int = 10) -> Tuple[bool, str]:
    with tempfile.TemporaryDirectory() as tmp:
        sol = pathlib.Path(tmp) / "solution.py"
        tst = pathlib.Path(tmp) / "test_solution.py"
        sol.write_text(solution_code, encoding="utf-8")
        tst.write_text(test_code, encoding="utf-8")

```

```

def run_py_tests(solution_code: str, test_code: str, timeout_sec: int = 10) -> Tuple[bool, str]:
    with tempfile.TemporaryDirectory() as tmp:
        sol = pathlib.Path(tmp) / "solution.py"
        tst = pathlib.Path(tmp) / "test_solution.py"
        sol.write_text(solution_code, encoding="utf-8")
        tst.write_text(test_code, encoding="utf-8")

        cmd = [sys.executable, str(tst)]
        try:
            proc = subprocess.run(cmd, capture_output=True, text=True, timeout=timeout_sec, cwd=tmp)
            out = (proc.stdout or "") + (proc.stderr or "")
            passed = proc.returncode == 0
            return passed, out.strip()
        except subprocess.TimeoutExpired as e:
            return False, f"TIMEOUT after {timeout_sec}s\n{e}"
        except Exception as e:
            return False, f"EXEC_ERROR: {e}"

async def _ask_async(agent: AssistantAgent, message: str, label: str) -> str:
    await asyncio.sleep(SLEEP_BETWEEN_CALLS_SEC)
    result = await agent.on_messages(
        messages=[TextMessage(content=message, source="user")],
        cancellation_token=None,
    )

```

```

        except Exception as e:
            return False, f"EXEC_ERROR: {e}"

    async def _ask_async(agent: AssistantAgent, message: str, label: str) -> str:
        await asyncio.sleep(SLEEP_BETWEEN_CALLS_SEC)
        result = await agent.on_messages(
            messages=[TextMessage(content=message, source="user")],
            cancellation_token=None,
        )

        content = ""
        if hasattr(result, "chat_message") and getattr(result.chat_message, "content", None) is not None:
            content = result.chat_message.content
        elif hasattr(result, "messages") and result.messages:
            content = getattr(result.messages[-1], "content", "") or ""
        else:
            content = str(result)

        log_event("llm_reply", {"agent": getattr(agent, "name", "assistant"), "label": label, "content": content})
        return content

    def ask(agent: AssistantAgent, message: str, label: str) -> str:
        return asyncio.run(_ask_async(agent, message, label))

```

3.2 Define the three agents (CodeGen, Tester, and Debugger) using the following code:

```

codegen = AssistantAgent(
    name="CodeGen",
    system_message=(
        "You are CodeGen. Write Python code for solution.py with a SINGLE\n"
        "function:\n"
        "def is_palindrome(text: str) -> bool\n"
        "- Return True if text is a palindrome.\n"
        "- Ignore non-alphanumeric characters and case.\n"
        "- Do not print; no comments; no extra text.\n"
        "Reply with ONLY a Python fenced code block for solution.py."
    ),
    model_client=model_client,
)

```

```

tester = AssistantAgent(
    name="Tester",
    system_message=(
        "You are Tester. Write tests in test_solution.py using Python asserts.\n"
        "Import from solution import is_palindrome.\n"
        "Cover typical and edge cases (empty, punctuation, mixed case, long string).\n"
        "Reply with ONLY a Python fenced code block for test_solution.py."
    ),
    model_client=model_client,
)

```

```

debugger = AssistantAgent(
    name="Debugger",
    system_message=(
        "You are Debugger. Given failing test output and the current solution.py, "
        "produce a FIXED solution.py that passes tests. Keep the same signature:\n"
        "def is_palindrome(text: str) -> bool\n"
        "Reply with ONLY a Python fenced code block for solution.py."
    ),
    model_client=model_client,
)

```

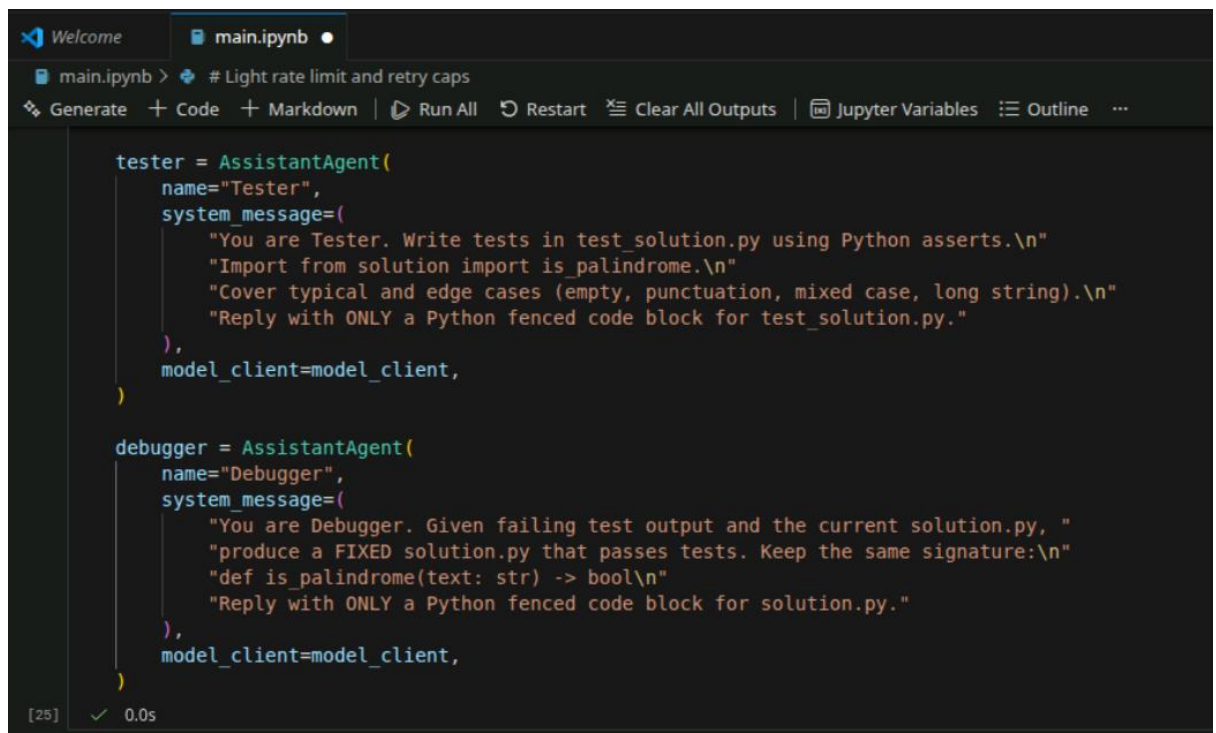
The screenshot shows a Jupyter Notebook window with a dark theme. The notebook has two tabs: 'Welcome' and 'main.ipynb'. The 'main.ipynb' tab is active, showing a code cell with Python code. The code defines two AssistantAgents: 'codegen' and 'tester'. The 'codegen' agent is configured to write Python code for a single function 'is_palindrome' that returns a boolean. The 'tester' agent is configured to write tests for the 'is_palindrome' function using Python asserts. Both agents are initialized with a 'model_client' parameter. The code is as follows:

```

codegen = AssistantAgent(
    name="CodeGen",
    system_message=(
        "You are CodeGen. Write Python code for solution.py with a SINGLE function:\n"
        "def is_palindrome(text: str) -> bool\n"
        "- Return True if text is a palindrome.\n"
        "- Ignore non-alphanumeric characters and case.\n"
        "- Do not print; no comments; no extra text.\n"
        "Reply with ONLY a Python fenced code block for solution.py."
    ),
    model_client=model_client,
)

tester = AssistantAgent(
    name="Tester",
    system_message=(
        "You are Tester. Write tests in test_solution.py using Python asserts.\n"
        "Import from solution import is_palindrome.\n"
        "Cover typical and edge cases (empty, punctuation, mixed case, long string).\n"
        "Reply with ONLY a Python fenced code block for test_solution.py."
    ),
    model_client=model_client,
)

```



```
tester = AssistantAgent(
    name="Tester",
    system_message=(
        "You are Tester. Write tests in test_solution.py using Python asserts.\n"
        "Import from solution import is_palindrome.\n"
        "Cover typical and edge cases (empty, punctuation, mixed case, long string).\n"
        "Reply with ONLY a Python fenced code block for test_solution.py."
    ),
    model_client=model_client,
)

debugger = AssistantAgent(
    name="Debugger",
    system_message=(
        "You are Debugger. Given failing test output and the current solution.py, "
        "produce a FIXED solution.py that passes tests. Keep the same signature:\n"
        "def is_palindrome(text: str) -> bool\n"
        "Reply with ONLY a Python fenced code block for solution.py."
    ),
    model_client=model_client,
)
```

[25] ✓ 0.0s

Step 4: Implement and run the orchestrator logic

4.1 Write and execute the main() function using the following code:

```
async def main():
    print("== AutoGen Code Testing and Debugging Chain (Azure OpenAI via APIM) ==")
    print(f"Deployment: {MODEL}")
    log_event("start", {"deployment": MODEL})

    # 1) CodeGen creates solution.py
    cg_out = await _ask_async(codegen, "Write solution.py as specified. Return ONLY the code block.", "codegen")
    solution = extract_code_block(cg_out)
    if "def is_palindrome" not in solution:
        cg_out = await _ask_async(codegen, "Please include def is_palindrome(text: str) -> bool", "codegen_retry")
        solution = extract_code_block(cg_out)
    log_event("solution", {"code": solution})
    print("\n[CodeGen] Generated solution.py")

    # 2) Tester creates test_solution.py
```

```
tt_out = await _ask_async(tester, "Create test_solution.py for is_palindrome;  
ONLY code block.", "tester")
```

```
tests = extract_code_block(tt_out)  
log_event("tests", {"code": tests})  
print("[Tester] Generated test_solution.py")
```

```
# 3) Execute tests
```

```
passed, output = run_py_tests(solution, tests, timeout_sec=10)  
print("\n[Test Runner] First run passed? ", passed)  
print(output)  
log_event("test_run", {"passed": passed, "output": output})
```

```
# 4) If failed, loop with Debugger
```

```
rounds = 0  
while not passed and rounds < MAX_DEBUG_ROUNDS:  
    rounds += 1  
    print(f"\n[Debugger] Fix attempt #{rounds}")  
    dbg_prompt = (  
        "Tests failed with output below. Current solution.py is also provided.\n\n"  
        f"=== FAIL LOG ===\n{output}\n\n=== CURRENT solution.py  
===\n``python\n{solution}\n``"  
    )  
    dbg_out = await _ask_async(debugger, dbg_prompt,  
f"debugger_round_{rounds}")  
    fixed = extract_code_block(dbg_out)  
    if fixed and fixed != solution:  
        solution = fixed  
    log_event("debug_fix", {"round": rounds, "code": solution})  
  
    passed, output = run_py_tests(solution, tests, timeout_sec=10)  
    print("[Test Runner] Passed after fix? ", passed)  
    print(output)  
    log_event("test_rerun", {"round": rounds, "passed": passed, "output":  
output})
```

```
# 5) Save artifacts and finish
```

```
(RUN_DIR / "solution.py").write_text(solution, encoding="utf-8")  
(RUN_DIR / "test_solution.py").write_text(tests, encoding="utf-8")  
status = "ALL TESTS PASSED" if passed else "STOPPED WITH FAILURES (see logs)"  
print("\n== RESULT:", status)  
print("Artifacts saved to:", RUN_DIR)
```

```
log_event("end", {"status": status, "artifacts_dir": str(RUN_DIR)})
```

```
await main()
```

```
main.ipynb •
main.ipynb > # Light rate limit and retry caps
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ...
venv (Python 3.12.3)

async def main():
    print("== AutoGen Code Testing and Debugging Chain (Azure OpenAI via APIM) ==")
    print(f"Deployment: {MODEL}")
    log_event("start", {"deployment": MODEL})

    # 1) CodeGen creates solution.py
    cg_out = await _ask_async(codegen, "Write solution.py as specified. Return ONLY the code block.", "codegen")
    solution = extract_code_block(cg_out)
    if "def is_palindrome" not in solution:
        cg_out = await _ask_async(codegen, "Please include def is_palindrome(text: str) -> bool", "codegen_retry")
        solution = extract_code_block(cg_out)
    log_event("solution", {"code": solution})
    print("\n[CodeGen] Generated solution.py")

    # 2) Tester creates test_solution.py
    tt_out = await _ask_async(test, "Create test_solution.py for is_palindrome; ONLY code block.", "tester")
    tests = extract_code_block(tt_out)
    log_event("tests", {"code": tests})
    print("[Tester] Generated test_solution.py")

    # 3) Execute tests
    passed, output = run_py_tests(solution, tests, timeout_sec=10)
    print("\n[Test Runner] First run passed? ", passed)
    print(output)
    log_event("test_run", {"passed": passed, "output": output})

    # 4) If failed, loop with Debugger
```

```
main.ipynb •
main.ipynb > async def main():
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ...
venv (Python 3.12.3)

# 3) Execute tests
passed, output = run_py_tests(solution, tests, timeout_sec=10)
print("\n[Test Runner] First run passed? ", passed)
print(output)
log_event("test_run", {"passed": passed, "output": output})

# 4) If failed, loop with Debugger
rounds = 0
while not passed and rounds < MAX_DEBUG_ROUNDS:
    rounds += 1
    print(f"\n[Debugger] Fix attempt #{rounds}")
    dbg_prompt = (
        "Tests failed with output below. Current solution.py is also provided.\n\n"
        f"=== FAIL LOG ===\n{output}\n\n=== CURRENT solution.py ===\n``python\n{solution}\n```"
    )
    dbg_out = await _ask_async(debugger, dbg_prompt, f"debugger_round_{rounds}")
    fixed = extract_code_block(dbg_out)
    if fixed and fixed != solution:
        solution = fixed
    log_event("debug_fix", {"round": rounds, "code": solution})

    passed, output = run_py_tests(solution, tests, timeout_sec=10)
    print("[Test Runner] Passed after fix? ", passed)
    print(output)
    log_event("test_rerun", {"round": rounds, "passed": passed, "output": output})
```

```
main.ipynb •
main.ipynb > async def main():
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ...
venv (Python 3.12.3)

log_event("debug_fix", {"round": rounds, "code": solution})

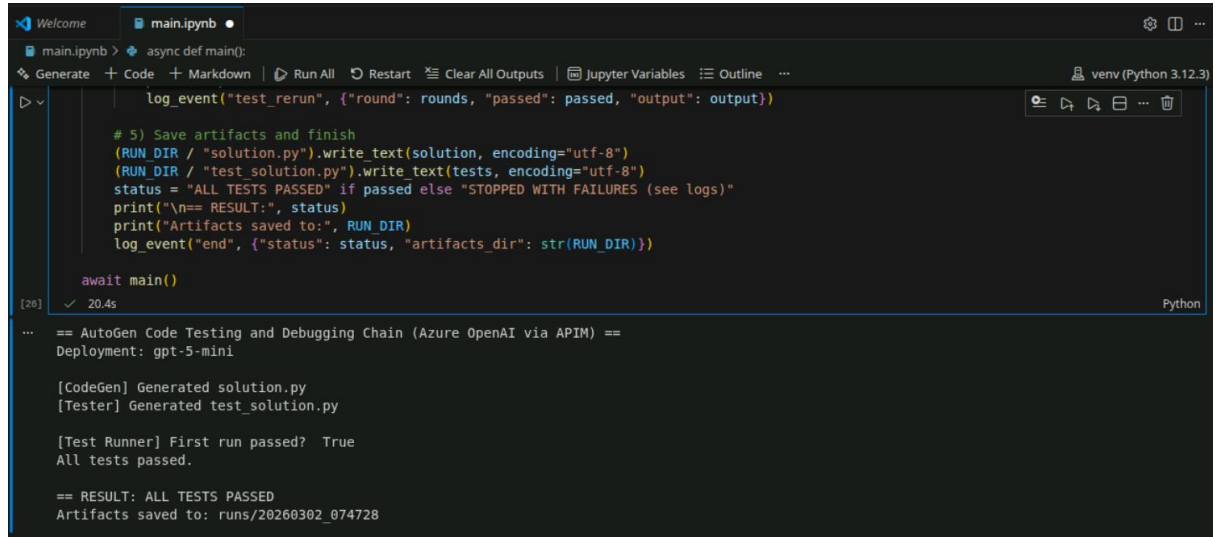
passed, output = run_py_tests(solution, tests, timeout_sec=10)
print("[Test Runner] Passed after fix? ", passed)
print(output)
log_event("test_rerun", {"round": rounds, "passed": passed, "output": output})

# 5) Save artifacts and finish
(RUN_DIR / "solution.py").write_text(solution, encoding="utf-8")
(RUN_DIR / "test_solution.py").write_text(tests, encoding="utf-8")
status = "ALL TESTS PASSED" if passed else "STOPPED WITH FAILURES (see logs)"
print("\n== RESULT: ", status)
print("Artifacts saved to:", RUN_DIR)
log_event("end", {"status": status, "artifacts_dir": str(RUN_DIR)})

await main()

[20] ✓ 20.4s Python
```

The following screenshot shows the successful run of the AutoGen code testing and debugging chain: CodeGen created solution.py, tester created test_solution.py, all tests passed on the first run, and artifacts were saved in a timestamped runs/ folder.



```
log_event("test_rerun", {"round": rounds, "passed": passed, "output": output})

# 5) Save artifacts and finish
(RUN_DIR / "solution.py").write_text(solution, encoding="utf-8")
(RUN_DIR / "test_solution.py").write_text(tests, encoding="utf-8")
status = "ALL TESTS PASSED" if passed else "STOPPED WITH FAILURES (see logs)"
print("\n== RESULT: ", status)
print("Artifacts saved to:", RUN_DIR)
log_event("end", {"status": status, "artifacts_dir": str(RUN_DIR)})

await main()
```

```
[26] ✓ 20.4s Python

... == AutoGen Code Testing and Debugging Chain (Azure OpenAI via APIM) ==
Deployment: gpt-5-mini

[CodeGen] Generated solution.py
[Tester] Generated test_solution.py

[Test Runner] First run passed? True
All tests passed.

== RESULT: ALL TESTS PASSED
Artifacts saved to: runs/20260302_074728
```

By following these steps, you have successfully built an AutoGen agent chain for automated code testing and debugging, gaining practical experience in setting up agents, configuring Azure OpenAI, running tests, and automating fixes for more reliable workflows.